

PHP basics

1. Introduction to PHP

PHP is a **server-side scripting language** used to build dynamic websites.

PHP can:

- Generate HTML dynamically
- Handle forms
- Work with user input
- Manage sessions and cookies
- Connect to databases later
- Build login systems, dashboards, blogs, and web apps

A PHP file ends with:

```
.php
```

Basic PHP code:

```
<?php  
echo "Hello, PHP!";  
?>
```

2. PHP Setup

To run PHP locally, you can use:

- XAMPP

A typical project folder:

```
php-workbook/  
|  
└─ index.php
```

Example file:

```
<?php  
echo "PHP is working!";
```

If you use XAMPP, place your project inside:

```
htdocs/
```

Then open in browser:

```
http://localhost/php-workbook/index.php
```

3. PHP Syntax

PHP code starts with:

```
<?php
```

Example:

```
<?php  
echo "Welcome to PHP";
```

Each statement usually ends with a semicolon:

```
echo "Hello";  
echo "World";
```

Comments

Single-line comment:

```
<?php
// This is a comment
echo "Learning PHP";
```

Multi-line comment:

```
<?php
/*
This is a multi-line comment.
It can explain a larger block of code.
*/
echo "PHP comments are useful.";
```

PHP with HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>PHP Page</title>
</head>
<body>

<h1>Welcome</h1>

<p>
  <?php
  echo "This text came from PHP.";
  ?>
</p>
```

```
</body>
</html>
```

PHP runs on the server, then sends the final HTML to the browser.

4. Variables

Variables store data.

In PHP, variables start with `$`.

```
<?php
$name = "Alex";
$age = 22;

echo $name;
echo $age;
```

Joining text and variables

Use the dot `.` for concatenation:

```
<?php
$name = "Alex";
$age = 22;

echo "Name: " . $name . "<br>";
echo "Age: " . $age;
```

Output:

```
Name: Alex
Age: 22
```

Variables inside double quotes

```
<?php
$name = "Maria";

echo "Hello, $name";
```

Output:

```
Hello, Maria
```

Single quotes vs double quotes

```
<?php
$name = "Maria";

echo "Hello $name <br>";
echo 'Hello $name';
```

Output:

```
Hello Maria
Hello $name
```

Double quotes process variables.

Single quotes print text more literally.

5. Data Types

PHP has several common data types.

Type	Example	Meaning
String	"Hello"	Text

Type	Example	Meaning
Integer	25	Whole number
Float	10.5	Decimal number
Boolean	true / false	Yes or no
Array	["Apple", "Mango"]	Multiple values
NULL	null	No value

Example

```
<?php
$name = "Rahim";
$age = 22;
$cgpa = 3.75;
$isStudent = true;
$middleName = null;

var_dump($name);
echo "<br>";

var_dump($age);
echo "<br>";

var_dump($cgpa);
echo "<br>";

var_dump($isStudent);
echo "<br>";

var_dump($middleName);
```

`var_dump()` shows both the value and data type.

Boolean example

```
<?php
$isLoggedIn = true;

if ($isLoggedIn) {
    echo "Welcome back!";
} else {
    echo "Please login.";
}
```

6. Operators

Operators are used to calculate, compare, and make decisions.

6.1 Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Remainder
**	Power

Example:

```
<?php
$a = 20;
$b = 6;

echo "Addition: " . ($a + $b) . "<br>";
echo "Subtraction: " . ($a - $b) . "<br>";
echo "Multiplication: " . ($a * $b) . "<br>";
echo "Division: " . ($a / $b) . "<br>";
```

```
echo "Remainder: " . ($a % $b) . "<br>";  
echo "Power: " . ($a ** 2);
```

6.2 Assignment Operators

```
<?php  
$score = 10;  
  
$score += 5;  
echo $score . "<br>"; // 15  
  
$score -= 3;  
echo $score . "<br>"; // 12  
  
$score *= 2;  
echo $score . "<br>"; // 24  
  
$score /= 4;  
echo $score . "<br>"; // 6
```

6.3 Comparison Operators

Operator	Meaning
<code>==</code>	Equal value
<code>===</code>	Equal value and same type
<code>!=</code>	Not equal
<code>!==</code>	Not equal value or not same type
<code>></code>	Greater than
<code><</code>	Less than
<code>>=</code>	Greater than or equal
<code><=</code>	Less than or equal

Example:

```
<?php
$a = 10;
$b = "10";

var_dump($a == $b);
echo "<br>";

var_dump($a === $b);
```

Output:

```
bool(true)
bool(false)
```

`==` checks only value.

`===` checks value and type.

6.4 Logical Operators

Operator	Meaning
<code>&&</code>	AND
<code>,</code>	
<code>!</code>	NOT

Example:

```
<?php
$username = "admin";
$password = "12345";

if ($username === "admin" && $password === "12345") {
    echo "Login successful";
} else {
```

```
    echo "Invalid login";  
}
```

7. Conditional Statements

Conditional statements help your program make decisions.

7.1 **if**

```
<?php  
$age = 18;  
  
if ($age >= 18) {  
    echo "You are an adult.";  
}
```

7.2 **if...else**

```
<?php  
$age = 16;  
  
if ($age >= 18) {  
    echo "You can vote.";  
} else {  
    echo "You cannot vote yet.";  
}
```

7.3 **if...elseif...else**

```
<?php  
$marks = 82;
```

```

if ($marks >= 80) {
    echo "Grade: A+";
} elseif ($marks >= 70) {
    echo "Grade: A";
} elseif ($marks >= 60) {
    echo "Grade: B";
} elseif ($marks >= 50) {
    echo "Grade: C";
} elseif ($marks >= 40) {
    echo "Grade: D";
} else {
    echo "Grade: F";
}

```

7.4 Nested condition

```

<?php
$username = "admin";
$password = "12345";

if ($username === "admin") {
    if ($password === "12345") {
        echo "Welcome admin.";
    } else {
        echo "Wrong password.";
    }
} else {
    echo "Unknown user.";
}

```

7.5 switch

Use `switch` when you compare one value with many possible cases.

```
<?php
$day = "Friday";

switch ($day) {
    case "Friday":
        echo "Weekend starts!";
        break;

    case "Saturday":
        echo "Rest day.";
        break;

    case "Sunday":
        echo "Work starts.";
        break;

    default:
        echo "Regular day.";
}
```

8. Loops

Loops repeat code.

8.1 **for** loop

Use when you know how many times to repeat.

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo "Number: $i <br>";
}
```

Output:

```
Number: 1  
Number: 2  
Number: 3  
Number: 4  
Number: 5
```

8.2 **while** loop

Use when repetition depends on a condition.

```
<?php  
$count = 1;  
  
while ($count <= 5) {  
    echo "Count: $count <br>";  
    $count++;  
}
```

8.3 **do...while** loop

Runs at least once.

```
<?php  
$count = 10;  
  
do {  
    echo "Count is $count <br>";  
    $count++;  
} while ($count <= 5);
```

Even though the condition is false, this still runs once.

8.4 **foreach** loop

Best for arrays.

```
<?php
$fruits = ["Apple", "Banana", "Mango"];

foreach ($fruits as $fruit) {
    echo $fruit . "<br>";
}
```

Robust Example: Multiplication Table

```
<?php
$number = 5;

for ($i = 1; $i <= 10; $i++) {
    echo "$number x $i = " . ($number * $i) . "<br>";
}
```

9. Arrays

Arrays store multiple values in one variable.

9.1 Indexed Array

```
<?php
$students = ["Ali", "Nora", "David"];

echo $students[0] . "<br>";
echo $students[1] . "<br>";
echo $students[2] . "<br>";
```

Indexes start from 0.

9.2 Loop through Indexed Array

```
<?php
$students = ["Ali", "Nora", "David"];

foreach ($students as $student) {
    echo "Student: $student <br>";
}
```

9.3 Associative Array

Associative arrays use named keys.

```
<?php
$student = [
    "name" => "Ali",
    "age" => 21,
    "department" => "CSE"
];

echo "Name: " . $student["name"] . "<br>";
echo "Age: " . $student["age"] . "<br>";
echo "Department: " . $student["department"];
```

9.4 Loop through Associative Array

```
<?php
$student = [
    "name" => "Ali",
    "age" => 21,
    "department" => "CSE"
];
```

```
foreach ($student as $key => $value) {  
    echo ucfirst($key) . ": $value <br>";  
}
```

Output idea:

```
Name: Ali  
Age: 21  
Department: CSE
```

9.5 Multidimensional Array

A multidimensional array is an array inside another array.

```
<?php  
$students = [  
    [  
        "name" => "Ali",  
        "marks" => 85  
    ],  
    [  
        "name" => "Nora",  
        "marks" => 92  
    ],  
    [  
        "name" => "David",  
        "marks" => 76  
    ]  
];  
  
foreach ($students as $student) {  
    echo $student["name"] . " scored " . $student["marks"] .  
    "<br>";  
}
```


Robust Example: Student Result Table

```
<?php
$students = [
    ["name" => "Ali", "marks" => 85],
    ["name" => "Nora", "marks" => 92],
    ["name" => "David", "marks" => 38],
    ["name" => "Sara", "marks" => 67]
];
?>

<!DOCTYPE html>
<html>
<head>
    <title>Student Results</title>
</head>
<body>

<h2>Student Results</h2>

<table border="1" cellpadding="10">
    <tr>
        <th>Name</th>
        <th>Marks</th>
        <th>Status</th>
    </tr>

    <?php foreach ($students as $student): ?>
        <tr>
            <td><?php echo $student["name"]; ?></td>
            <td><?php echo $student["marks"]; ?></td>
            <td>
                <?php
                if ($student["marks"] >= 40) {
                    echo "Pass";
                }
            </td>
        </tr>
    </?php>
</table>
```

```

        } else {
            echo "Fail";
        }
    ?>
</td>
</tr>
<?php endforeach; ?>

</table>

</body>
</html>

```

10. Functions

Functions are reusable blocks of code.

10.1 Basic Function

```

<?php
function sayHello() {
    echo "Hello from a function!";
}

sayHello();

```

10.2 Function with Parameter

```

<?php
function greetUser($name) {
    echo "Hello, $name!";
}

```

```
greetUser("Maria");
```

10.3 Function with Return Value

```
<?php
function addNumbers($a, $b) {
    return $a + $b;
}

$result = addNumbers(10, 20);

echo "Result: $result";
```

10.4 Function with Type Hints

```
<?php
function calculateTotal(float $price, int $quantity): float {
    return $price * $quantity;
}

echo calculateTotal(250.50, 3);
```

This makes the function clearer and safer.

10.5 Default Parameter Value

```
<?php
function greet($name = "Guest") {
    echo "Hello, $name!";
}
```

```
greet();  
echo "<br>";  
greet("Amina");
```

Output:

```
Hello, Guest!  
Hello, Amina!
```

Robust Example: Grade Calculator Functions

```
<?php  
function getGrade(int $marks): string {  
    if ($marks >= 80) {  
        return "A+";  
    } elseif ($marks >= 70) {  
        return "A";  
    } elseif ($marks >= 60) {  
        return "B";  
    } elseif ($marks >= 50) {  
        return "C";  
    } elseif ($marks >= 40) {  
        return "D";  
    } else {  
        return "F";  
    }  
}  
  
function getStatus(int $marks): string {  
    return $marks >= 40 ? "Pass" : "Fail";  
}  
  
$students = [  
    ["name" => "Ali", "marks" => 85],
```

```
[ "name" => "Nora", "marks" => 72],  
[ "name" => "David", "marks" => 35]  
];  
  
foreach ($students as $student) {  
    echo $student["name"] . " - ";  
    echo "Grade: " . getGrade($student["marks"]) . " - ";  
    echo "Status: " . getStatus($student["marks"]) . "<br>";  
}
```